

Riwaz Shrestha

Collaborators: Denali Schlesinger

HW 7 Write-up

Q1:

In my graphs file, I import all the libraries I will use.

I create a Graph struct to represent the given file. This struct contains a hashmap of usize to represent the vertices and a vec of usize to contain all of the vertices it is connected to.

In the implementation, I have a function to create the struct and add_edge, which adds new entries or edits entries that already exist.

My data_to_graph function takes in a file name and returns a graph.

In the function, I access and open the file using File and BufReader.

Then, for each line, not counting the first (the number of vertices), I split it by the space, collect it in a vector, and then add it as an edge in my graph variable.

This function returns a Graph struct.

I also have a Vertex struct that contains node, termination, and rank. This was created as a struct to store pagerank and calculate the top 5.

This struct has an implementation with a new function to create a new instance and a swap_with function to swap the contents inside with another struct of Vertex.

In the main file, I import all necessary libraries and then create the page_rank function, where I iterate through the hashmap keys and make a clone of the keys and store it in a vector. I also have a termination map to keep count of how many times each vertex is terminated on. In each iteration, I loop 90 times to simulate 90 walks. In each walk, I loop 90 times to simulate the 90 steps. In each step, I randomly pick a number 1-10. If the number is 1 or 2, I move to a completely random key in the hashmap. If the number is 3-10, I move to a random vertex that the current one is connected to. After 90 steps, I map the termination point in the termination map.

I then iterate through the termination map and get the top 5 nodes with the largest termination values. I return the top 5 as a sorted vector of Vertices.

Result:

Vertex: 596, approximate PageRank: 0.0020333333

Vertex: 603, approximate PageRank: 0.0018222223

Vertex: 78, approximate PageRank: 0.0014111111

Vertex: 135, approximate PageRank: 0.0013888889

Vertex: 502, approximate PageRank: 0.0013777778

Q2:

In the tree file, I imported rng to use random.

I created a Node struct to represent each node in the tree. It contains left and right Option enums to contain its children. It implements set_left and set_right to set its children.

My build_tree function makes the root, and while the number of nodes is less than 120, it generates a random number and creates that number of children. Its children recursively call the Node struct to create more children of their own. I keep a vector of nodes and refer back to it to check for the number of nodes in the tree.

My diameter_of_tree function recursively runs BFS and returns the largest number of edges in each side.

In my main file, I build the tree and run diameter_of_tree on it.

My test function runs the test on a smaller tree.

Result:

Diameter of the tree is: 23

The only place I used ChatGPT in this homework was to figure out how to insert two children into a parent at the same time. It gave the ".extend" function. Everything else was found using prior homeworks, the lecture notes, and the rust book.